

HelixAgent

المحتوى

HelixAgent

لا تختبر نموذجًا واحدًا — دعهم يتناقشون، وقدم الإجابة التي يتفقون عليها

الملخص

تجمع بين Go، في AI تجميعية جاهزة للإنتاج، مدعومة بنظام LLM هو خدمة HelixAgent متعددة الجولات واختيار مقدمي الخدمة AI استجابات عدة نماذج لغوية — بما في ذلك نظام مناظرات ديناميكيًا بناءً على التحقق — لإنتاج المخرجات الأكثر دقة وموثوقية.

وصف مختصر

تجمع بين عدة مقدمي خدمة في إجابة Go تجميعية قائمة على LLM هي خدمة HelixAgent متعددة الجولات، وتقيم مقدمي الخدمة ديناميكيًا عبر AI واحدة دقيقة. تعمل بنظام مناظرات وتوجه الطلبات باستراتيجيات موزونة بالثقة، وتوفر ميزات إنتاجية: التخزين، LLMsVerifier، OpenAI، المؤقت، والمراقبة، وضوابط الأمان، وواجهات برمجة تطبيقات على نمط

وصف مفصل

HelixAgent (MIT ترخيص) AI تجميعية جاهزة للإنتاج، مدعومة بنظام LLM هي خدمة HelixAgent تعامل إجابة النموذج الواحد كقرصية، لا حكمًا نهائيًا. بدلاً من المراهنة على مقدم خدمة واحد قد يكون مخطئًا أو متحيزًا أو غير متاح مؤقتًا، تجمع الخدمة بين استجابات عدة نماذج لغوية لتتقارب نحو المخرجات الأكثر دقة وموثوقية — وعندما يكون السؤال معقدًا بما يكفي، تخضع النماذج لمناظرة منظمة العديد من مقدمي خدمة README متعددة الجولات. تضم القائمة مجموعة واسعة: يوثق ملف LLM من مقدمي خدمة Claude، وDeepSeek، بما في ذلك internal/llm/providers/، Gemini، وMistral، وQwen، وxAI/Grok، و

الأهم من ذلك أن اختيار مقدم الخدمة ليس قائمة تفضيلات ثابتة، بل يُكتسب في الوقت الفعلي. تقود المدمج توجيه الطلبات والعودة التدريجية إلى LLMsVerifier درجات التحقق المباشرة من نظام الخلاف AI أفضل مقدم أداء، مع تقارير الأخطاء المصنفة عند تدهور أداء أحدهم. يحول منظم مناظرات → إلى إشارة: يدعم عدة بنيات (شبكة، نجمية، سلسلة) وبروتوكول مراحل منضبط — الاقتراح → النقد المراجعة → التركيب — مع تعلم مشترك بين المناظرات لتحسين قدرة النظام على التوفيق بين النماذج

بمرور الوقت. تشمل استراتيجيات التوجيه الاختيار الموزون بالثقة، وإجماع الأغلبية، والكشف عن القصد الدلالي، كلها مع بث مباشر للاستجابات بحيث تصل الإجابات رمزياً برمجياً تلو الآخر بدلاً من انتظار استقرار المجموعة بأكملها.

PostgreSQL صُممت الخدمة لتعمل في بيئات الإنتاج، لا لمجرد العرض التوضيحي: يشكل Grafana و Prometheus طبقة بيانات عالية التوافر، بينما يوفر Redis و OpenTelemetry المصادقة، وتحديد المعدل JWT المقاييس ولوحات التحكم والتتبع، وتحيط OpenTelemetry والتجميع لضمان التحكم الذي تتطلبه النشر الفعلي. تنظم الخدمة في PII ومحركات الضوابط، وكشف والمراقبة، والمصادقة، والتخزين، وقواعد البيانات المتجهية، EventBus (نحو عشرين وحدة مستقلة وغيرها)، كل منها مسؤولية منفصلة، وتوفر إطار عمل تحسين MCP، والذاكرة، و RAG، والتضمينات، و SGLang مع تكاملات لـ (التخزين المؤقت الدلالي، والمخرجات المنظمة، والبث المحسن) LLM. ولأن نقاط نهاية الإكمال والتجميع LLMQL، و Guidance، و LangChain، و LlamaIndex، للحصول على HelixAgent يمكن للعميل الحالي توجيه طلباته إلى OpenAI متوافقة مع استدلال تجميعي دون الحاجة لإعادة كتابة الكود.

المحتوى

لماذا أنشأناها

لتمكين HelixAgent واحدة، أو تكون متحيزة، أو غير متاحة. تم بناء LLM قد تخطئ أي التطبيقات من استشارة عدة نماذج في آن واحد، وتقييم إجاباتها بناءً على موثوقيتها المقاسة، والعودة إلى بدائل بشكل سلس—مما يحول الاعتماد الهش على مزود واحد إلى مجموعة مرنة ذات تقييم ذاتي

لماذا تُعد ثورة في المجال

تعمل على إضفاء الطابع العملي على توافق النماذج المتعددة—نقل عملية “أسأل عدة نماذج وصالح بينها” من نصوص برمجية مؤقتة إلى خدمة إنتاجية. فبدلاً من ترميز مزود واحد والتمني للأفضل، تحصل الفرق على توجيه ديناميكي مدفوع بنتائج التحقق الفوري، وبروتوكول منظم للنقاش في الأسئلة التي لا تكفيها محاولة واحدة، ومرونة إنتاجية عالية (طبقة بيانات عالية التوفر، ومراقبة كاملة، وضوابط أمان)—كل الميزة الحقيقية هي التبني دون تعطيل: يتحول API و OpenAI ذلك خلف واجهة متوافقة مع الاعتماد الهش على مزود واحد إلى مجموعة مرنة ذات تقييم ذاتي، ويمكن للعملاء الحاليين التبديل إليها. بتغيير نقطة النهاية بدلاً من تعديل الكود.

ما الجديد فيها

- يعامل الخلاف بين النماذج كمورد: طوبولوجيات قابلة للاختيار AI نقاش منظم متعدد الجولات لـ و بروتوكول منضبط للمراحل (اقترح → نقد → مراجعة → تركيب)، وتعلم (شبكة/نجمة/سلسلة) عبر النقاشات يتراكم مع الوقت.
- الحية بدلاً من قائمة تفضيلات LLMsVerifier اختيار ديناميكي للمزودين بناءً على درجات ثابتة—فالمجموعة توجه الطلبات إلى من يؤدي بشكل أفضل في الوقت الحالي، وتعود إلى بدائل بشكل سلس عند تراجع الأداء.
- قائم (ذاكرة تخزين دلالية، إخراج منظم، بث محسن) LLM و Go إطار عمل أصلي لتحسين (SGLang، LlamaIndex، بذاته، مع إمكانية إضافة محسنات خارجية اختيارية عند الحاجة بدلاً من اشتراطها) LLMQL، Guidance، LangChain.
- بنية معيارية مكونة من نحو عشرين وحدة منفصلة تحافظ على فصل الاهتمامات وتفتح الباب أمام ميزات البيانات الضخمة مثل الذاكرة الموزعة وبث الرسوم البيانية المعرفية.

أكبر التحديات التقنية وكيف تغلبنا عليها

- يختلف المزودون في الجودة ويتغير أدائهم مع الوقت، لذا فإن اختيار بين مزودين غير متساوين أي ترتيب ثابت يصبح غير صالح بحلول الغد. حللنا هذه المشكلة بجعل الاختيار خاضعاً للقياس تغذي توجيه الطلبات بناءً على ثقة مرجحة وتصويت LLMsVerifier المستمر: فمرجات الأغلبية، مع عودة سلسلة إلى البدائل بحيث يتم تجنب المزود المتدهور بدلاً من الوثوق به.
- النموذج الواحد، عند سؤاله مرة واحدة، لا يملك الحصول على إجابة موثوقة للأسئلة الصعبة حقاً آلية لاكتشاف خطأه. يوفر منظم النقاش حلاً لذلك—من خلال نقاش متعدد الطوبولوجيات ومراحل متسلسلة (اقترح → نقد → مراجعة → تركيب) يجبر النماذج على تحدي بعضها وتحسين الإجابات قبل تركيب الإجابة النهائية.
- توزيع الطلبات على عدة مزودين. تشغيل مجموعة في الإنتاج، وليس فقط في دفتر الملاحظات ومراقبة PostgreSQL+Redis. يضاعف فرص الفشل. احتويناه بطبقة بيانات عالية التوفر، لكشف أي خلل في المزود أو المسار Prometheus/Grafana/OpenTelemetry PII. وتحديد معدلات الطلبات، ومحرك ضوابط، وكشف، JWT وحاجز أمني يشمل مصادقة

المحتوى

مجموعة الأدوات التقنية

- اختيرت لأن إرسال طلب واحد إلى عدة مقدمي خدمات بشكل متزامن هو بالضبط ما — **Go** كما أن النشر في ملف ثنائي واحد يبقي الخدمة المكونة من نحو، غوروثينات صُممت من أجله ال وحدة بسيطة في النشر؛ وهي تشكل الأساس الذي تعتمد عليه الخدمة بأكملها وكل وحدة داخلية 20 فيها.
- سريعة ومنخفضة العبء؛ فهي تقدم نقاط HTTP اختيرت لتوفير واجهة — **Gin (ويب API)** والدردشة والبث والتجميع، مما يسمح للعملاء v1 / للإكمال **OpenAI** النهاية المتوافقة مع الحاليين باعتماد التجميع دون تغيير.
- اختيرت لتكون المخزن الدائم للجلسات والتحليلات وسجلات — **PostgreSQL** المناقشات، بحيث تكون قرارات التوافق وسجل المناقشات قابلة للتدقيق؛ وهي تشكل ركيزة طبقة البيانات عالية التوفر.
- اختيرت للتخزين المؤقت منخفض الكمون وإدارة قوائم المهام؛ فهي تدعم التخزين — **Redis** المؤقت للاستجابات وطبقة التخزين الدلالي التي تسمح بتجاوز الاستدلال المتكرر أو شبه المتكرر عند تكرار الطلبات أو تشابهها.
- اختيرت لتحويل موثوقية مقدمي الخدمات إلى كمية قابلة — **LLMsVerifier (الدمجة)** للقياس بدلاً من افتراضها؛ حيث تصنف درجاتها مقدمي الخدمات للتوجيه وتدير التراجع عند تدهور أداء أحدهم.
- اختيرت لضمان قابلية — **Prometheus + Grafana + OpenTelemetry** helixagent_* مراقبة التجميع الذي يشمل العديد من مقدمي الخدمات؛ فهي تعرض مقاييس ولوحات التحكم وتتبع الطلبات من البداية إلى النهاية عبر التوزيع المتزامن.
- اختيرت لتمكين قابلية التوسع — **Model Context Protocol (MCP)** محولات لربط الأدوات MCP العديد من محولات README عبر بروتوكول مفتوح؛ حيث يسرد ملف الخارجية والسياقات.
- اختيرت للتجاوز حدود — **Neo4j / ClickHouse / Kafka (البيانات الضخمة)** الذاكرة الموزعة وميزات الرسم البياني Neo4j و ClickHouse العقد الواحدة: حيث يدعم ببث تلك البيانات البيانية وبيانات الأحداث على نطاق واسع Kafka للمعرفة، بينما يقوم
- اختيرت لإضافة التخزين المؤقت للبادئات والاسترجاع وتحليل — **SGLang, LlamaIndex, LangChain, Guidance, LMQL** عمليات التكامل لتحسين الأداء

المهام والتوليد المقيد كخدمات اختيارية، بحيث يكون التحسين الأكثر تعقيداً متاحاً دون أن يكون إلزامياً.

ملاحظات حول الحالة والشفافية

- تُوصف الخدمة بأنها جاهزة للإنتاج، لكن أرقام الأداء والتغطية المذكورة في ملف .الحالة: تجريبية مثل “أكثر من 1000 طلب في الثانية”، “أقل من 500 مللي ثانية للمخزن) README المؤقت”، وعدد مقدمي الخدمات ونصوص التحقق) هي ادعاءات المشروع الذاتية وغير مدققة بشكل مستقل، وقد تم الاحتفاظ بها هنا بشكل نوعي عن قصد.
- نفسه؛ حيث يستخدم النص صياغة نوعية README يتفاوت عدد مقدمي الخدمات داخل ملف “هي” العديد من مقدمي الخدمات

أساسية-Helix: درجة الأولوية